

P-2: Smart API Rate-Limit & Abuse Simulator

In modern software systems, many users or systems use the same service at the same time. To keep the service reliable and fair, the system internally keeps track of how requests are made and checks whether the usage is normal or abnormal.

Build a JavaFX-based desktop application that simulates how modern web systems protect APIs by:

- Tracking client request activity
- Applying rate-limiting rules
- Detecting abnormal or abusive usage patterns
- Generating a usage and violation report per client

This project models concepts used in cloud platforms, fintech systems, and large-scale web services, without requiring real networking or servers. The project simulates service request behavior using manually generated or file-based data. No real service, API, or network communication is required or expected. In this project, you are NOT building a real online service, login system, or payment system. You are building a simulation that models how such systems internally record requests and analyze usage patterns.

Tools & Technologies:

- Java 21+
- JavaFX (GUI)
- Java Collections (Map, List)
- Java Time API (LocalDateTime, Duration)
- Optional: File I/O (import/export request logs)

Core OOP Concepts:

- Domain Modeling (Client, RequestLog, RatePolicy)
- Encapsulation of counters and thresholds
- Polymorphism (different rate-limit policies)
- Strategy-like design (policy evaluation)
- Use of Enums (RequestType, ViolationLevel)

Suggested Class Structure:

- Client (clientId, name)
- RequestLog (timestamp, requestType)
- RatePolicy (interface: evaluate(Client))
- FixedWindowPolicy implements RatePolicy
- SlidingWindowPolicy implements RatePolicy
- AbuseReport (violations, severity)

Features Students Must Implement:

- Register API clients
- Record simulated API requests
- Apply rate-limit policies
- Detect excessive requests within time windows
- Generate per-client abuse report

2-Week Timeline (Flexible):

Week 1: Design + Logging Logic

Day 1–2: API usage discussion, abuse scenarios

Day 3–4: Class and policy design

Day 5–6: JavaFX UI (client, request entry)

Day 7: Request logging and storage

Week 2: Policy Evaluation + Reporting

Day 8–9: Implement rate-limit rules

Day 10: Violation classification

Day 11–12: Edge-case testing

Day 13–14: UI polish and submission

Evaluation Rubric (Out of 100):

Functional GUI and logging workflow 25

Policy-based OOP design 25

Correct rate-limit detection 20

Use of polymorphism and interfaces 10

Report clarity and correctness 10

Code quality and documentation 10

Total 100

1. What This Project Simulates

This project simulates how a **software service** (e.g., authentication service, payment service, search service) handles **incoming requests from clients**, tracks usage behavior, and applies **usage control rules**.

It does **not** simulate:

- networking
- HTTP
- REST
- real servers

It simulates **logical behavior only**, using objects and collections.

Think of it as a **request log + rule engine**.

2. Core Simulation Idea

“A service receives requests.

Each request is recorded.

The service periodically analyzes request history and decides whether usage is normal or abusive.”

Everything happens **inside memory**, using Java objects.

3. What Kind of Data Is Simulated?

3.1 Service Request Data

Each request represents **one logical service usage action**.

Example real-world equivalents:

- Login attempt
- Data fetch
- Payment request
- Profile update

Simulated Data Attributes

Attribute	Data Type	Meaning
timestamp	LocalDateTime	When request happened
requestType	enum	Type of request
clientId	String	Who made the request

4. How to Generate the Data

You may generate request data in **three allowed ways** (choose at least one):

Option A: Manual Entry via JavaFX Form

UI fields:

- Client selection (ComboBox)
- Request type (ComboBox)
- “Add Request” button

When button is clicked:

- A `ServiceRequest` object is created
- Timestamp is auto-generated
- Stored in a collection

Option B: Predefined Test Dataset

You can:

- Load a `.txt` or `.csv` file
- Parse each line into `ServiceRequest` objects

Example text record:

10:15:30, CLIENT_A, READ

This supports **repeatable testing**.

Option C: Simulated Burst Generator

A button such as:

“Generate 20 Requests in 10 Seconds”

Internally:

- Loop creates multiple request objects
- Timestamp increases artificially

This helps test rate-limit logic.

5. Data Flow

Step-by-Step Flow

```
User Action
↓
JavaFX Form Input
↓
ServiceRequest Object Created
↓
Stored in Collection (List or Map)
↓
Analyzer Processes Requests
↓
Usage Report Generated
```

No background threads.

No listeners.

No networking.